From Unit IV

HTML + CSS + TypeScript
↓
Angular
↓
Structured Web Application

TypeScript → Programming language
Angular → Web application framework

5.1 Introduction to Angular, Features, Angular CLI and Project Setup:

Need for Angular

Large applications contain many features:

Login
Products
Cart
Payment
Dashboard
Profile

Without proper structure:

Large Application
↓
More Code
↓
Repeated Code
↓
Difficult Maintenance

Angular provides a structured solution.

Definition: Angular is a web application framework used to develop structured, interactive, and scalable client-side applications. It commonly uses TypeScript, HTML, and CSS and provides built-in support for components, data binding, forms, routing, services, and HTTP communication.

Definition: A framework is a collection of predefined structures, libraries, tools, and conventions that provides an organized foundation for developing applications.

Without Framework
→ Developer organizes everything manually

With Angular

→ Framework provides structure + tools

Definition: A Single-Page Application (SPA) loads the main application initially and changes required views dynamically without loading a completely new HTML document for every interaction.

Angular Working

HTML + CSS + TypeScript

↓

Angular

↓

Components + Templates

Data Binding + Services

Forms + Routing + HTTP

↓

Structured Web Application

Main Angular Features

Components

→ Reusable UI parts

Templates

→ HTML interface

Data Binding

→ Connect component and UI

Directives

→ Control template behaviour

Pipes

→ Format displayed data

Services

→ Reusable logic

Dependency Injection

→ Supplies services

Forms

→ User input

Routing

→ Navigation

HTTP

→ Server communication

Angular CLI
→ Project management

Definition: Angular CLI is a command-line development tool used to create, configure, run, build, and manage Angular projects using ng commands.

commands:
ng version
→ Check Angular CLI

ng new project-name
→ Create project

ng serve
→ Run project

ng build
→ Build project

ng generate
→ Generate Angular code

Definition: An Angular project is a structured collection of source files, configuration files, dependencies, and resources used to develop an Angular application.

Project Setup
node --version
npm --version
npm install -g @angular/cli
ng version
ng new college-portal
cd college-portal
ng serve

Open: http://localhost:4200/

Definition: ng serve builds the Angular application for development and starts a local development server so the application can be tested in a browser.

Node.js
↓
npm
↓
Angular CLI

↓

ng new college-portal

↓

Angular Project

↓

cd college-portal

↓

ng serve

↓

Local Development Server

↓

Browser

Essential Project Files

src

→ Application source files

package.json

→ Dependencies + scripts

angular.json

→ Angular configuration

tsconfig.json

→ TypeScript configuration

node_modules

→ Installed packages

Summary

Angular

→ Web application framework

Framework

→ Ready structure + tools

SPA

→ Dynamic view changes

Angular CLI

→ Command-line development tool

ng new

→ Create project

ng serve

→ Run project

5.2 Components and Modules

1. Connection with 5.1

In 5.1, we created and ran an Angular application.

"Should the whole website be written as one large block?"

No.

Large Angular Application

↓

Divide UI into Small Parts

↓

Components

↓

Organize Related Features

↓

Modules / Standalone Structure

Notes define components as the fundamental UI building blocks of Angular applications.

2. Need for Components

A shopping website may contain:

Header

Product List

Product Card

Cart

Login

Footer

If everything is written together:

Large UI

↓

Large Code

↓

Difficult to Understand

↓

Difficult to Reuse

↓

Difficult to Maintain

Angular solves this by dividing the UI into components.

Application

↓

Small Components

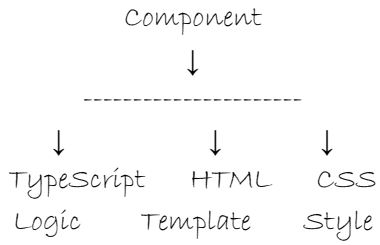
↓

Reusable Parts

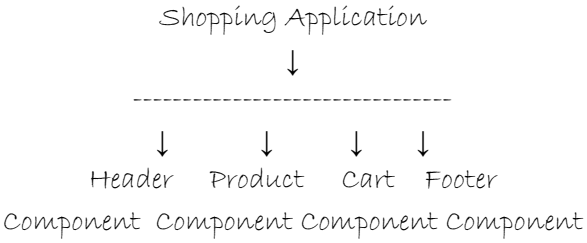
↓

Better Organization

3. Definition: A component is a fundamental building block of an Angular application that controls a specific section of the user interface. It normally contains a TypeScript class, a template, and optional styles.



4. Example of Components



Each component has one UI responsibility.

5. Component Syntax

```
import {
   Component
} from '@angular/core';

@Component({
   selector: 'app-product',
   template: `
      <h2>Product Details</h2>
   `
})
export class ProductComponent {
}
```

Important parts:

@Component

→ Component metadata

selector

→ Component HTML name

template

→ Component UI

ProductComponent

→ TypeScript class

6. Definition: @Component is an Angular decorator that provides metadata describing how a TypeScript class should work as an Angular component.

This Class

↓

Is an Angular Component

↓

Use this Selector

↓

Use this Template

7. Definition: A component class is the TypeScript class that contains the data and behaviour of an Angular component.

Example:

```
export class ProductComponent {

  productName: string =
    "Laptop";

  productPrice: number =
    50000;
}
```

Memory:

Component Class

→ Data

→ Methods

→ Application Logic

8. Definition: A selector identifies where Angular should create and display a component inside a template.

Example:

selector: 'app-product'

Use:

<app-product></app-product>

Flow:

selector

'app-product'

↓

<app-product>

↓

Angular Finds Component

↓

Component Displayed

9. Definition: A template is the HTML-based user interface associated with an Angular component.

Example:

```
template: `
   <h2>Product Details</h2>
   <p>Laptop</p>
`
```

Simple memory:

Component Class

→ Logic

Template

→ User Interface

10. Inline vs External Template

Inline

```
template: `
   <h2>Product</h2>
`
```

External

```
templateUrl:
   './product.component.html'
```

Relationship:

product.component.ts

↓

templateUrl

↓

product.component.html

↓

User Interface

11. Component Styles: Component styles define the CSS used for the presentation of a component.

Example:

```
styles: [`
   h2 {
      font-size: 24px;
   }
`]
```

So:

Component

↓

Logic + Template + Styles

12. Typical Component Files

Angular CLI may create:

```
product/
 |
 ├──── product.component.ts
 ├──── product.component.html
 ├──── product.component.css
 └──── product.component.spec.ts
```

Remember:

.ts

→ Logic

.html

→ Template

.css

→ Style

.spec.ts

→ Test file

13. Creating Component Using CLI

Command:

```
ng generate component product
```

Short form:

```
ng g c product
```

Memory:

ng g c

→ Generate Component

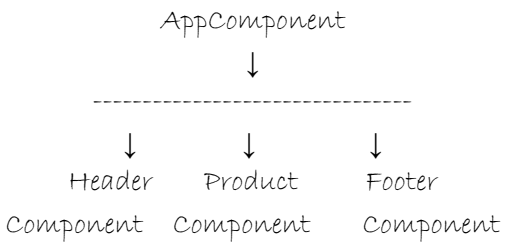
14. Using One Component Inside Another

Example:

```
<app-header></app-header>

<app-product></app-product>

<app-footer></app-footer>
```



15. Parent and Child Components

If one component contains another:

AppComponent

↓

ProductComponent

Then:

AppComponent

→ Parent

ProductComponent

→ Child

16. Definition: A standalone component is an Angular component that can participate in an application without first being declared inside an NgModule.

Example:

```
@Component({
  selector: 'app-product',
  standalone: true,
  template: `
    <h2>Product</h2>
  `
})
export class ProductComponent {
}
```

Important memory:

Modern Angular

→ Often uses standalone components

Standalone Component

→ No NgModule declaration required

17. Definition: An Angular NgModule is a class decorated with @NgModule that groups related Angular components, directives, pipes, and dependencies into a logical unit.

NgModule

↓

Groups Angular Features

Strong diagram:

Angular Module

↓

↓ ↓ ↓

Components Directives Pipes

18. Definition: @NgModule is a decorator that provides metadata describing how an Angular module is organized.

syntax:

```
@NgModule({
  declarations: [],
  imports: [],
  exports: []
})
export class AppModule {
}
```

19. NgModule Metadata

declarations

Identifies non-standalone components, directives, and pipes that belong to the module.

```
declarations: [
  ProductComponent
]
```

imports

Identifies required modules or supported standalone dependencies.

```
imports: [
  CommonModule
]
```

exports

Makes selected declarations available to other modules.

```
exports: [
  ProductComponent
]
```

providers

Configures dependencies that Angular's dependency injection system can supply.

For this topic, remember only:

declarations

→ Belongs to module

imports

→ Use dependencies

exports

→ Share members

providers

→ Dependency configuration

20. NgModule Example

```
import {
  NgModule
} from '@angular/core';

import {
  ProductComponent
} from './product.component';

@NgModule({
  declarations: [
    ProductComponent
  ],
  exports: [
    ProductComponent
  ]
})
export class ProductModule {
}
```

Working:

ProductComponent
↓
Declared in
↓
ProductModule
↓
Exported
↓
Can Be Used Elsewhere

21. Creating a Module Using CLI

Command:

ng generate module product

Short form:

ng g m product

Memory:

ng g m

→ Generate Module

22. Component vs Module

Component	NgModule
Represents UI section	Groups Angular functionality
Uses @Component	Uses @NgModule
Has logic + template	Has declarations/imports/exports
Visible in UI	Mainly organizational
UI building block	Feature organization

Component
→ Builds UI

Module
→ Organizes Angular Features

23. TypeScript Module vs Angular NgModule

This is important because Unit IV already had modules.

TypeScript Module

```
export class Product {
}
```

Uses:
export
import

Angular NgModule

```
@NgModule({
})
export class ProductModule {
}
```

Difference:

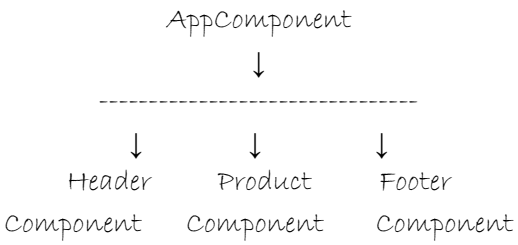
TypeScript Module
→ File-based organization
→ import / export

Angular NgModule

→ Angular metadata grouping
→ @NgModule

24. Complete Website Scenario

Shopping application:



Product component:

```
import {
   Component
} from '@angular/core';
```

```
@Component({
  selector: 'app-product',
  standalone: true,
  template: `
    <h2>Product Details</h2>
    <p>Product: Laptop</p>
    <p>Price: ₹50000</p>
  `
})
export class ProductComponent {
}
```

Use:

```
<app-product></app-product>
```

Flow:

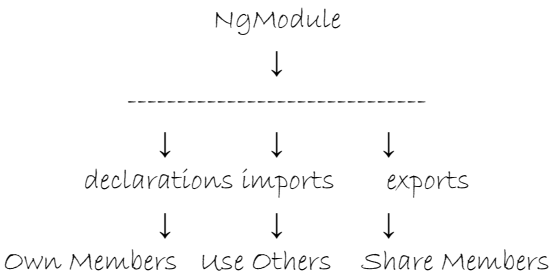
ProductComponent
↓
selector
↓
app-product
↓
<app-product>
↓
Angular Renders
↓
Product UI

25. Diagram

Angular Application
↓
Components
↓

↓ ↓ ↓
Logic Template Style
.ts .html .css
↓
@Component
↓
Selector
↓
Rendered UI

For module-based applications:



26. Summary

Component

→ UI building block

@Component

→ Component metadata

selector

→ Component HTML name

template

→ Component interface

templateUrl

→ External HTML file

ng g c

→ Generate component

Then:

NgModule

→ Organizes Angular functionality

@NgModule

→ Module metadata

declarations

→ Module members

imports

→ Required dependencies

exports

→ Shared members

ng g m

→ Generate module

And:

Standalone Component

→ No NgModule declaration required

The summary and modern standalone-vs-NgModule distinction are explicitly emphasized in your notes.

5.3 Templates and Data Binding

1. Connection with 5.2

In 5.2, we created a component.

"The component has data and logic, but how will the user see that data and interact with it?"

Component

↓

Contains Data + Logic

↓

Data Binding

↓

Template

↓

User Interface

2. Definition: An Angular template is the HTML-based user interface associated with a component. It can contain normal HTML together with Angular syntax for displaying data, setting properties, handling events, and working with user input.

Template

→ What the user sees

Example:

```
<h2>{{ productName }}</h2>
```

3. Inline and External Template

Inline Template

```
@Component({
  selector: 'app-product',
  template: `
    <h2>Product Details</h2>
  `
})
```

External Template

```
@Component({
  selector: 'app-product',
  templateUrl:
    './product.component.html'
})
```

product.component.ts

↓

templateUrl

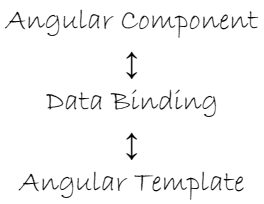
↓

product.component.html

↓

User Interface

4. Definition: Data binding is the mechanism that connects an Angular component with its template and allows information to move between application logic and the user interface.



5. Four Types of Data Binding

{{ }}

→ Interpolation

[ ]

→ Property Binding

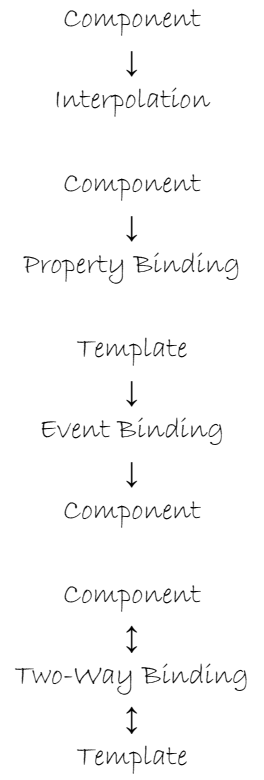
( )

→ Event Binding

[()]

→ Two-Way Binding

Main direction:



6. Definition: Interpolation is used to display the value of a component expression as text in a template. It uses double curly braces {{ }}.

Syntax:

```
{{ expression }}
```

```
Component:
productName: string =
  "Laptop";
Template:
<h2>
  {{ productName }}
</h2>
```

Output: Laptop

{{ }}
→ Display Text

7. Interpolation with Expression

Component:

```
productPrice: number = 500;
quantity: number = 2;
```

```
Template:
<p>
  Total:
  {{ productPrice * quantity }}
</p>
```

Output:
Total: 1000

8. Definition: Property binding sets an HTML element, DOM object, component, or directive property using a value from the component. It uses square brackets [ ].

Syntax:

```
[property]="componentExpression"
```

Example:

```
productImage: string =
  "assets/laptop.jpg";
<img [src]="productImage">
```

Working:

Component
productImage
↓
Property Binding
↓

[src]

↓

Image Source

[ ]

→ Set Property

9. Property Binding with disabled

Component:

```
productAvailable: boolean = false;
```

Template:

```
<button[disabled]="!productAvailable">

  Buy Now

</button>
```

Working:

productAvailable = false

↓

!productAvailable = true

↓

disabled = true

↓

Button Disabled

10. Interpolation vs Property Binding

Interpolation	Property Binding
Displays text	Sets element/component property
Uses {{ }}	Uses [ ]
{{ productName }}	[src]="imageUrl"

{{ }}

→ Display

[ ]

→ Set

11. Definition: Event binding executes component logic when an event occurs in the user interface. It uses parentheses ( ).

Syntax:

```
(event)="method()"
```

Component:

```
showMessage(): void {
  console.log(
    "Product selected"
```

```
  );
}
```

Template:

```
<button
  (click)="showMessage()">
  Select Product
</button>
```

User Clicks
↓
(click)
↓
showMessage()
↓
Component Method Executes

( )
→ Respond to Event

12. Event Binding Website Example

```
Component:
message: string = "";

addToCart(): void {

  this.message =
     "Product added to cart";
}

Template:
<button
  (click)="addToCart()">

  Add to Cart

</button>

<p>
  {{ message }}
</p>
```

Working:

Click
↓
addToCart()
↓

message changes

↓

Interpolation updates

↓

Message displayed

13. Definition: Two-way data binding keeps a component property and a suitable user-interface value synchronized in both directions. Angular commonly uses [(ngModel)] for this purpose.

Syntax:

```
[(ngModel)]="componentProperty"
```

Example:

```
quantity: number = 1;
<input
  type="number"
  [(ngModel)]="quantity">

<p>
  Quantity:
  {{ quantity }}
</p>
```

Component

quantity

↕

[(ngModel)]

↕

Input Box

[()]

→ Synchronize Both Ways

14. Why FormsModule?

To use ngModel, forms support must be available.

Standalone component:

```
import {
  FormsModule
} from '@angular/forms';
```

Then:

```
@Component({
  imports: [
    FormsModule
  ]
})
```

FormsModule

↓

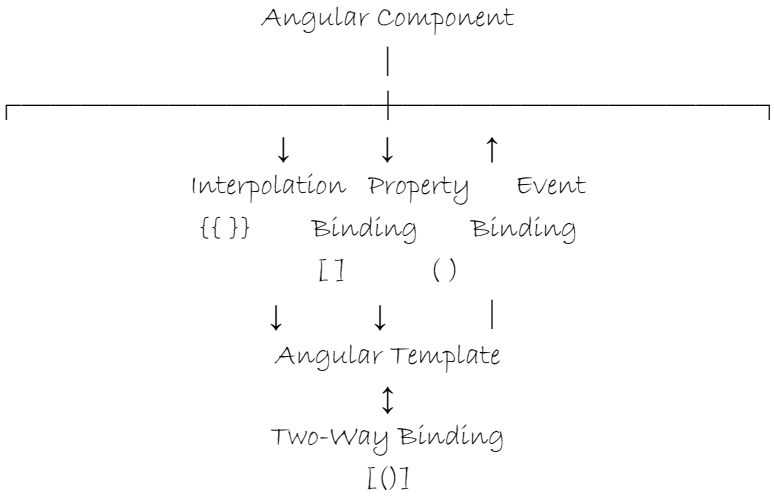
Provides ngModel

↓

Two-Way Binding

15. Binding Diagram

This is the main diagram students should remember:



16. Scenario — Product Page

Use one product page to connect all four bindings:

Product Name

→ Interpolation

Product Image

→ Property Binding

Add to Cart Click

→ Event Binding

Quantity Input

→ Two-Way Binding

17. Example

```
import {
  Component
} from '@angular/core';

import {
  FormsModule
} from '@angular/forms';

@Component({
  selector: 'app-product',

  standalone: true,

  imports: [
```

```
    FormsModule
  ],

  template: `
    <h2>
      {{ productName }}
    </h2>

    <p>
      Price:
      {{ productPrice }}
    </p>

    <img
      [src]="productImage"
      width="200">

    <br><br>

    <input
      type="number"
      [(ngModel)]="quantity">

    <p>
      Quantity:
      {{ quantity }}
    </p>

    <p>
      Total:
      {{ productPrice * quantity }}
    </p>

    <button
      [disabled]="availableStock === 0"
      (click)="addToCart()">

      Add to Cart

    </button>

    <p>
      {{ message }}
    </p>
  `
})
export class ProductComponent {
```

```
  productName: string = "Laptop";
  productPrice: number =50000;
  productImage: string ="assets/laptop.jpg";
  availableStock: number = 10;
  quantity: number = 1;
   message: string = "";
   addToCart(): void {
      this.message =this.quantity +" item(s) added to cart";
   }
}
```

This is the same combined beginner scenario used in your notes.

18. Program Mapping

productName

↓

{{ }}

↓

Display Product Name

productImage

↓

[src]

↓

Display Image

Button Click

↓

(click)

↓

addToCart()

quantity

↕

[(ngModel)]

↕

Input Box

19. Summary

Template

→ User Interface

Data Binding

→ Connects Component and Template

Then:

{{ }}
→ Interpolation
→ Display

[ ]
→ Property Binding
→ Set

( )
→ Event Binding
→ Respond

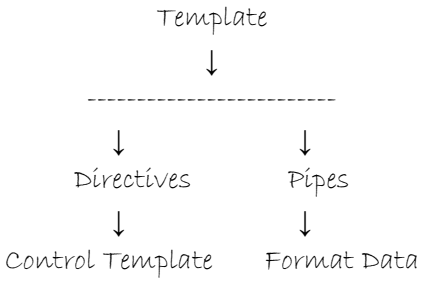
[()]
→ Two-Way Binding
→ Synchronize
And:
FormsModule
→ Required for ngModel

5.4 Directives and Pipes

1. Connection with 5.3

In 5.3, we connected component data with the template using data binding.

"How can the template show/hide content, repeat items, change styles, and format values?"



2. Definition: A directive is an Angular feature that changes template structure, behaviour, or the appearance of an existing element.

Directive

→ What should the template DO?

Examples:

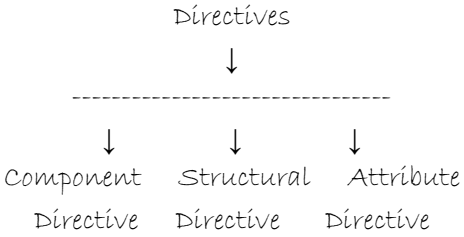
*ngIf

*ngFor

ngClass

ngStyle

3. Types of Directives



For this topic, mainly remember:

Structural

→ Changes template structure

Attribute

→ Changes appearance / behaviour

4. Definition: A structural directive changes the rendered template structure by adding, removing, or repeating content.

Traditional examples:

*ngIf

*ngFor

The * symbol is traditionally used with these directives.

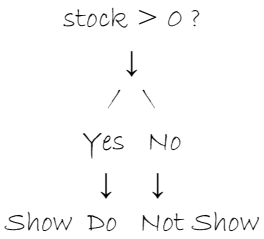
5. Definition: *ngIf is a structural directive used to conditionally display template content based on an expression.

Syntax:

```
<p *ngIf="condition">
  Content
</p>
```

Example:

```
stock: number = 10;
<p *ngIf="stock > 0">
  In Stock
</p>
```



6. Definition: *ngFor is a structural directive used to repeat template content for each item in a collection.

Component:

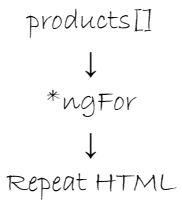
```
products: string[] = [
  "Laptop",
  "Mouse",
  "Keyboard"
];
```

Template:

```
<p *ngFor="let product of products">
  {{ product }}
</p>
```

Output:

Laptop
Mouse
Keyboard



for each item

7. *ngFor with Index

```
<p *ngFor="
    let product of products;
    let i = index ">
  {{ i + 1 }}.
  {{ product }}
</p>
```

Output:

1. Laptop
2. Mouse
3. Keyboard

8. Definition: An attribute directive changes the appearance or behaviour of an existing element without changing the basic template structure.

Examples:

ngClass
ngStyle

9. Definition: ngClass dynamically adds or removes CSS classes according to component data or expressions.

Syntax:

```
<element
  [ngClass]="expression">
</element>
```

Example:

```
productAvailable: boolean = true;
<p
  [ngClass]="{
    'available': productAvailable
  }">
  Product Status
</p>
```

Condition
↓
ngClass
↓
CSS Class Applied
↓
Appearance Changes

10. Definition: ngStyle dynamically applies inline style properties based on component data or expressions.

Example:

```
textSize: string = "24px";
<h2
  [ngStyle]="{
    'font-size': textSize
  }">
  Product Details
</h2>
```

ngClass

→ Change CSS class

ngStyle

→ Change style property

11. Modern Angular Control Flow

Modern Angular may use:

@if

@for

instead of traditional:

*ngIf

*ngFor

Example:

```
@if (productAvailable) {
  <p>In Stock</p>
}
@for (product of products; track product) {
  <p>{{ product }}</p>
}
```

Traditional

→ *ngIf, *ngFor

Modern Angular

→ @if, @for

12. Definition: A pipe is an Angular template feature used to transform a value into a required presentation format before it is displayed.

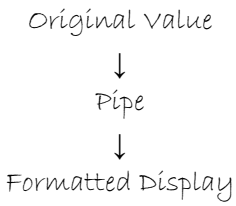
The original component value normally does not change. Only the displayed representation changes.

Syntax:

```
{{ value | pipeName }}
```

symbol: |

→ Pipe operator



13. uppercase Pipe

```
productName: string =
   "wireless mouse";
{{ productName | uppercase }}
```

Output:

WIRELESS MOUSE

14. lowercase Pipe

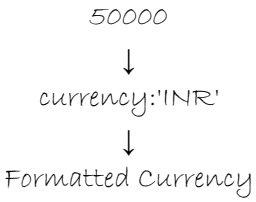
```
{{ courseName | lowercase }}
```

Example output:

web programming

15. currency Pipe

```
productPrice: number =
   50000;
{{ productPrice | currency:'INR' }}
```



16. date Pipe

```
today: Date = new Date();
{{ today | date:'dd/MM/yyyy' }}
```

Example display:

05/10/2026

17. percent Pipe

```
discountRate: number = 0.10;
{{ discountRate | percent }}
```

Output:10%

18. Common Built-In Pipes

uppercase

→ Uppercase text

lowercase

→ Lowercase text

currency

→ Currency format

date

→ Date format

percent

→ Percentage format

19. Pipe with Argument

Syntax:

{{ value | pipeName:argument }}

Examples:

{{ productPrice | currency:'INR' }}

{{ today | date:'dd/MM/yyyy' }}

20. Directives vs Pipes

Directives	Pipes
Control template behaviour/structure	Transform displayed values
Show, hide, repeat, style	Format text, price, date, percentage
*ngIf, *ngFor, ngClass	uppercase, currency, date

Directive

→ What should the template DO?

Pipe

→ How should the data LOOK?

21. CommonModule

For many classic built-in directives and pipes in a standalone component:

```
import {
  CommonModule
} from '@angular/common';
```

Then:

```
@Component({
  imports: [
    CommonModule
  ]
})
```

CommonModule

↓

Provides Common

Directives + Pipes

22. Scenario — Product Catalogue

A product page needs:

Repeat products

→ *ngFor

Show stock status

→ *ngIf

Apply CSS class

→ ngClass

Apply inline style

→ ngStyle

Format product name

→ uppercase

Format price

→ currency

Format discount

→ percent

Product Data

↓

Directives

↓

Control How Content Appears

↓

Pipes

↓

Format Displayed Values

23. Example

```
import {
  Component
} from '@angular/core';

import {
  CommonModule
} from '@angular/common';

@Component({
```

```
selector:
  'app-product-list',

standalone: true,

imports: [
  CommonModule
],

template: `
  <h2>
    Product Catalogue
  </h2>

  <div
    *ngFor="
      let product of products;
      let i = index
    ">

    <h3>
      {{ i + 1 }}.
      {{ product.name | uppercase }}
    </h3>

    <p>
      Price:
      {{
        product.price
        | currency:'INR'
      }}
    </p>

    <p>
      Discount:
      {{
        product.discount
        | percent
      }}
    </p>

    <p
      *ngIf="product.stock > 0"
      [ngClass]="{
        'available':
        product.stock > 0
      }">
```

```
          In Stock:
          {{ product.stock }}

        </p>

        <p
          *ngIf="product.stock === 0"
          [ngStyle]="{
            'font-weight':
            'bold'
          }">

          Out of Stock

        </p>

      </div>
  `
})
export class ProductListComponent {

  products = [
    {
      name: "Laptop",
      price: 50000,
      stock: 10,
      discount: 0.10
    },
    {
      name: "Wireless Mouse",
      price: 800,
      stock: 0,
      discount: 0.05
    }
  ];
}
```

24. Program Mapping

products

↓

*ngFor

↓

Repeat Products

stock

↓

*ngIf

↓

Show Correct Status

stock

↓

ngClass / ngStyle

↓

Change Appearance

name

↓

uppercase

↓

Formatted Text

price

↓

currency

↓

Formatted Price

discount

↓

percent

↓

Formatted Percentage

25. Summary

Directive

→ Controls template behaviour

*ngIf

→ Show conditionally

*ngFor

→ Repeat content

ngClass

→ Change CSS class

ngStyle

→ Change inline style

Then:

Pipe

→ Formats displayed data

|

→ Pipe symbol

uppercase

→ Uppercase text

currency

→ Currency

date

→ Date format

percent

→ Percentage

5.5 Services and Dependency Injection

1. Connection with 5.4

In 5.4, templates became dynamic using directives and pipes.

"What if many components need the same data or logic?"

Without a service:

ProductComponent

→ Product logic

CartComponent

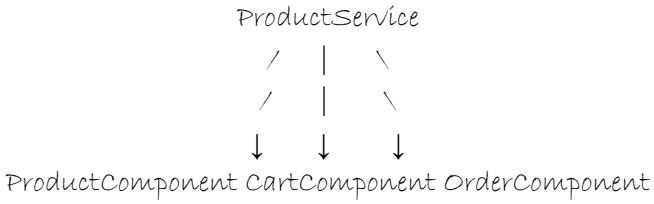
→ Same product logic

OrderComponent

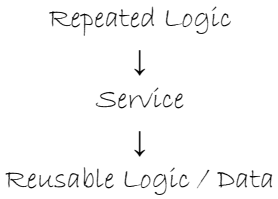
→ Same product logic

This creates duplication.

Better:



So:



2. Definition: A service is a reusable TypeScript class that contains data, application logic, or functionality that can be shared by different parts of an Angular application.

Service

→ Reusable Data / Logic

Typical uses:

Product Data

Cart Logic

User Information

Calculations

Server Communication

Shared State

3. Why Do We Need Services?

Without service:

Component A

→ Same Logic

Component B

→ Same Logic

Component C
→ Same Logic

With service:

Common Logic
↓
Service
↓
Used by Many Components

Advantages:
Code Reuse
Separation of Responsibility
Easy Maintenance
Shared Functionality
Less Duplication

4. Definition: @Injectable is an Angular decorator that marks a class as participating in Angular's dependency injection system.

Example:

```
import {
  Injectable
} from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ProductService {
}
```

@Injectable
→ Service participates in DI

5. Definition: providedIn: 'root' registers the service with Angular's root-level dependency injection environment so that it is generally available throughout the application.

ProductService
↓
@Injectable
↓
providedIn: 'root'
↓
Angular DI System
↓
Available to Application

6. Creating a Service Using CLI

Command:

ng generate service product

Short form:

ng g s product

Memory:

ng g s

→ Generate Service

7. Definition: A dependency is an object or service required by another class to perform its work.

Example:

ProductComponent

needs

ProductService

Therefore:

ProductService

→ Dependency of ProductComponent

8. Definition: Dependency Injection is a mechanism in which required dependencies are supplied to a class externally instead of being manually created inside that class.

Service

→ What is needed

Dependency Injection

→ How Angular supplies it

Component

↓

Requests Service

↓

Angular DI

↓

Provides Service

↓

Component Uses Service

9. Definition: An injector is part of Angular's dependency injection system that resolves and supplies requested dependencies.

Working:

Component Requests Service

↓

Angular Injector

↓

Find Provider

↓

Obtain Service Instance

↓

Provide to Component

10. Definition: A provider tells Angular how a requested dependency should be created or supplied.

Example:

```
@Injectable({
  providedIn: 'root'
})
export class ProductService {
}
```

Provider

→ Tells Angular how to supply dependency

11. Definition: Constructor injection is a technique in which a required dependency is declared as a constructor parameter and Angular supplies that dependency when creating the class.

Syntax:

```
constructor(
  private productService:
    ProductService
) {
}
```

ProductComponent

↓

Constructor Requests

ProductService

↓

Angular DI

↓

Supplies ProductService

12. Why Not Use new Manually?

Manual creation:

```
const service = new ProductService();
```

Here:

Component

→ Creates Service Itself

With DI:

```
constructor(
  private service:
    ProductService
) {
```

```
}
```

Here:

Component Requests Service

↓

Angular Supplies It

Comparison:

Manual new

→ Component controls creation

Dependency Injection

→ Angular supplies dependency

DI is the preferred Angular approach.

13. Service Example

product.service.ts

```
import {
  Injectable
} from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ProductService {

  getProductName(): string {
    return "Wireless Mouse";
  }
}
```

14. Using Service in Component

product.component.ts

```
import {
  Component
} from '@angular/core';

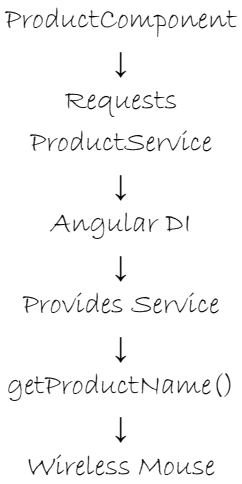
import {
  ProductService
} from './product.service';

@Component({
  selector: 'app-product',
  standalone: true,
  template: `
    <h2>
      {{ productName }}
    </h2>
```

```
  `
})
export class ProductComponent {
  productName: string;
  constructor(
    private productService:
      ProductService
  ) {

    this.productName =
      this.productService
        .getProductName();
  }
}
```

Working:



This is the same service-component pattern used in your notes.

15. Sharing One Service Between Components

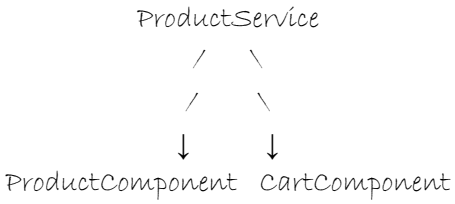
Suppose:

ProductComponent

CartComponent

both need product data.

Instead of repeating logic:



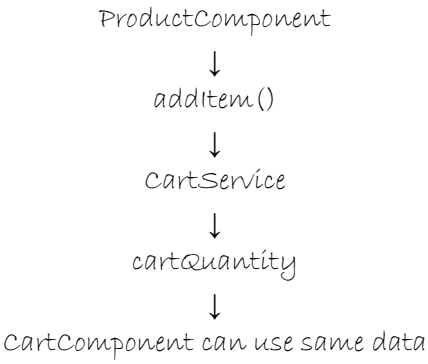
Both can request the same root-provided service.

16. Shared Data Example

```
@Injectable({
  providedIn: 'root'
})
```

```
export class CartService {
  cartQuantity: number = 0;
  addItem(): void {
    this.cartQuantity++;
  }
}
```

Concept:



A service can therefore contain both:

Reusable Logic

+

Shared Data

17. Service vs Component

Component	Service
Controls UI section	Contains reusable logic/data
Uses @Component	Uses @Injectable
Has template	Usually no UI template
Handles presentation	Handles reusable functionality

Component

→ User Interface

Service

→ Reusable Logic / Data

18. Service vs Dependency Injection

Do not confuse them.

Service

→ What is being reused?

Dependency Injection

→ How does component receive it?

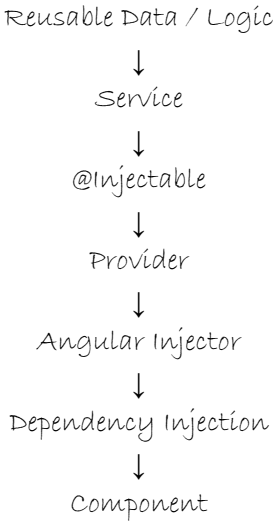
Example:

ProductService

→ Service

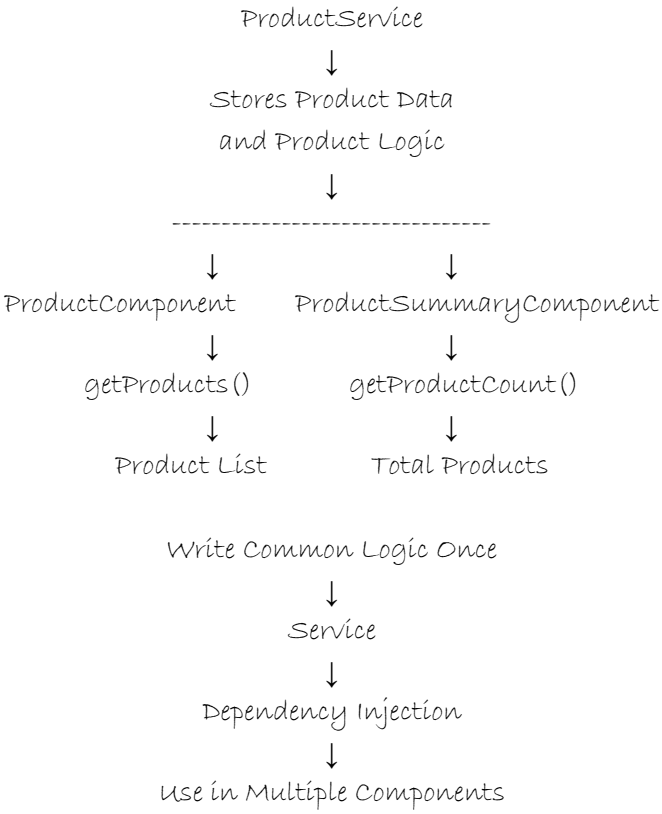
Angular supplies ProductService
to ProductComponent
→ Dependency Injection

19. Diagram



20. Scenario

Online store:



21. Modern inject() — Brief Note

Modern Angular also supports:

```
private productService =
  inject(ProductService);
```

But for this syllabus, remember constructor injection clearly:

```
constructor(
   private productService:
      ProductService
) {
}
```

Both use Angular's DI system.

22. Summary

Service

→ Reusable Data / Logic

@Injectable

→ Enables DI metadata

providedIn: 'root'

→ Root-level availability

Dependency

→ Required service/object

Provider

→ How dependency is supplied

Injector

→ Resolves dependency

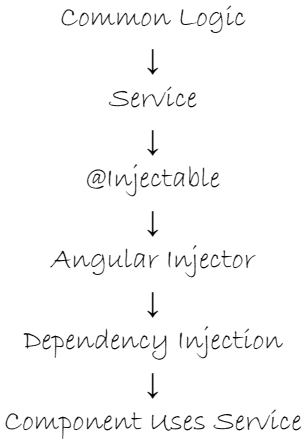
Dependency Injection

→ Angular supplies dependency

Constructor Injection

→ Request service in constructor

Diagram



5.6 Lifecycle Hooks and HTTP Client

1. Connection with 5.5

In 5.5, components used services through dependency injection.

"When should initialization code run, and how does Angular get data from a server?"

Component

↓

Lifecycle Stages

↓

Lifecycle Hooks

Component

↓

Service

↓

HttpClient

↓

Server / API

2. Definition: The component lifecycle is the sequence of stages through which an Angular component passes from creation to destruction.

diagram:

Component Created

↓

Inputs Initialized

↓

Component Initialized

↓

View Initialized

↓

Updates Checked

↓

Component Destroyed

3. Definition: A lifecycle hook is a specially named method that Angular calls at a particular stage of a component or directive lifecycle.

Main purpose:

Lifecycle Stage

↓

Angular Calls Hook

↓

Developer Code Runs

Important hooks:

ngOnChanges()

ngOnInit()

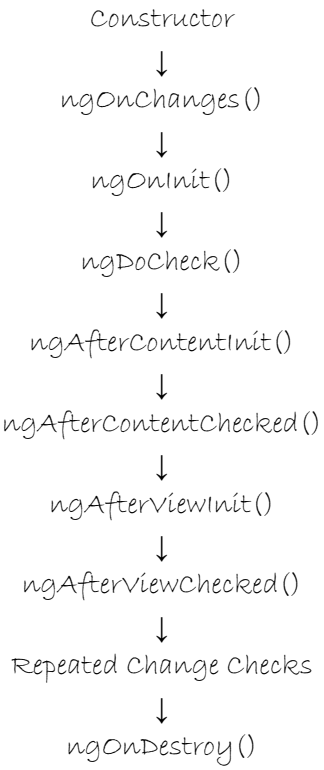
ngDoCheck()

ngAfterContentInit()

ngAfterContentChecked()

ngAfterViewInit()
ngAfterViewChecked()
ngOnDestroy()

4. Lifecycle Sequence



Remember mainly:
ngOnInit()
→ Start / Initialize

ngOnDestroy()
→ End / Cleanup

5. Constructor vs ngOnInit()

Constructor: A constructor is a TypeScript class feature that runs when the class object is created.

ngOnInit(): ngOnInit() is an Angular lifecycle hook that runs once after Angular initializes the component's initial inputs.

Difference:
constructor
→ Create class instance
→ Often used for dependency injection

ngOnInit()
→ Initialize component logic
→ Often used to load initial data

Important:

Constructor is not an Angular lifecycle hook.

6. Definition: ngOnInit() is a lifecycle hook used for initialization and initial data-loading operations.

Syntax:

```
ngOnInit(): void {

  this.loadData();
}
```

Scenario:

Student Opens Page
↓
Component Created
↓
ngOnInit()
↓
Load Student Data

7. Definition: ngOnChanges() runs when one or more input-bound properties of a component change.

Input Value Changes
↓
ngOnChanges()
↓
Component Responds

8. Definition: ngAfterViewInit() runs once after Angular initializes the component's view and child views.

View Ready
↓
ngAfterViewInit()

Use it when an operation requires the view to already exist.

9. Definition: ngOnDestroy() runs immediately before Angular destroys a component or directive and is used for cleanup.

Syntax:

```
ngOnDestroy(): void {

  console.log(
    "Component destroyed"
  );
}
```

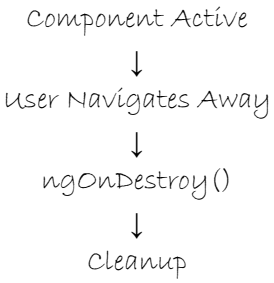
Typical cleanup:

Clear Timers

Remove Manual Event Handlers

Clean Long-Lived Resources

Diagram:



10. Lifecycle Hooks

Hook	Purpose
ngOnChanges()	Input changed
ngOnInit()	Initialize / load data
ngDoCheck()	Custom checking
ngAfterContentInit()	Content initialized
ngAfterContentChecked()	Content checked
ngAfterViewInit()	View ready
ngAfterViewChecked()	View checked
ngOnDestroy()	Cleanup

11. Need for HTTP Communication

Real Angular applications need current data from servers.

Examples:

Product List

Student Information

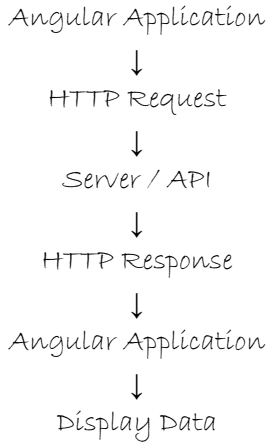
Notices

Orders

Weather

News

General flow:



12. Definition: HttpClient is an Angular service used to communicate with servers through HTTP requests.

Common methods:
get()
post()
put()
patch()
delete()

For this topic, mainly remember:
GET
→ Retrieve Data

13. Configuring HttpClient

Modern standalone Angular setup commonly uses:

```
import {
  provideHttpClient
} from '@angular/common/http';

export const appConfig = {
  providers: [
    provideHttpClient()
  ]
};
```

Meaning:

provideHttpClient()
↓
Configure HTTP Services
↓
HttpClient Available

14. Injecting HttpClient into a Service

```
constructor(
  private http: HttpClient
) {
}
```

Working:

Service
↓
Requests HttpClient
↓
Angular Dependency Injection
↓
Provides HttpClient

15. Definition: An HTTP GET request retrieves data from a specified web resource. In Angular, HttpClient.get() is commonly used for this purpose.

Syntax:

```
this.http.get<Type>(
  url
);
```

Example:

```
this.http.get<Notice>(
  'https://jsonplaceholder.typicode.com/posts/1'
);
```

16. Typed HTTP Response

Interface:

```
export interface Notice {

  userId: number;
  id: number;
  title: string;
  body: string;
}
```

Then:

```
this.http.get<Notice>(
  url
);
```

diagram:

HTTP JSON Response

↓

Notice Interface

↓

Typed Application Data

This connects to TypeScript interfaces and generics from Unit IV.

17. Definition: An Observable is an RXJS object that represents values that may become available over time. Angular HttpClient methods return Observables for asynchronous HTTP operations.

HttpClient.get()

↓

Observable

↓

subscribe()

↓

Receive Response

18. Definition: subscribe() attaches logic that receives values or errors emitted by an Observable.

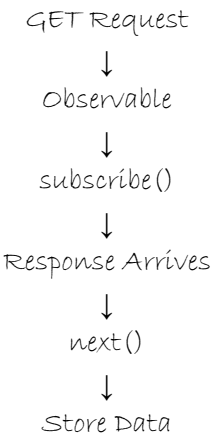
Example:

```
this.noticeService
  .getNotice()
  .subscribe({

    next: (data) => {
      this.notice = data;
    }

  });
```

Working:



19. Error Handling

```
this.noticeService
  .getNotice()
  .subscribe({

    next: (data) => {
      this.notice = data;
    },

    error: () => {
      this.message =
        "Unable to load notice";
    }

  });
```

next

→ Success

error

→ Failure

20. Why Put HTTP Code in a Service?

Service:

```
@Injectable({
```

```
  providedIn: 'root'
})
export class NoticeService {
  constructor(
    private http: HttpClient
  ) {
  }
  getNotice() {
    return this.http
      .get<Notice>(
        this.url
      );
  }
}
```

Reason:

Component

→ UI / Display

Service

→ HTTP Communication

This keeps responsibilities separated.

21. HTTP Working Diagram

Configure HttpClient

↓

Inject HttpClient into Service

↓

http.get()

↓

HTTP Request

↓

Server / API

↓

HTTP Response

↓

Observable

↓

subscribe()

↓

Component Data

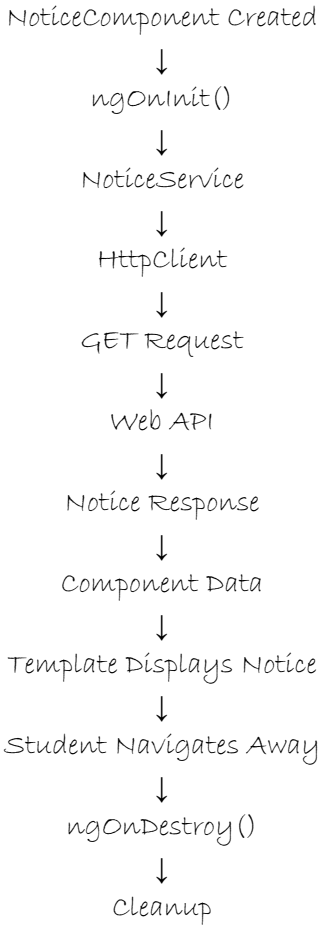
↓

Template

22. Scenario — College Notice

Student Opens Notice Page

↓



23. Example

notice.service.ts

```
@Injectable({
  providedIn: 'root'
})
export class NoticeService {

  private url =
    'https://jsonplaceholder.typicode.com/posts/1';

  constructor(
    private http: HttpClient
  ) {
  }

  getNotice() {

    return this.http
      .get<Notice>(
        this.url
      );
  }
}
```

notice.component.ts

```
export class NoticeComponent
  implements OnInit, OnDestroy {

  notice:
    Notice | null = null;

  constructor(
    private noticeService:
      NoticeService
  ) {
  }

  ngOnInit(): void {

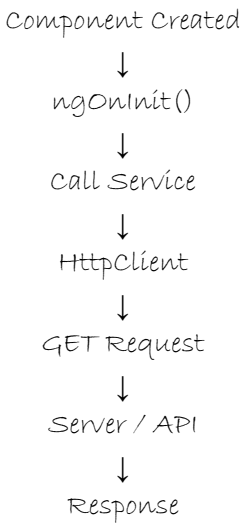
    this.noticeService
      .getNotice()
      .subscribe({

        next: (data) => {
          this.notice = data;
        }

      });
  }
  ngOnDestroy(): void {

    console.log(
      "Notice component destroyed"
    );
  }
}
```

24. Diagram



↓

Observable

↓

subscribe()

↓

Update Component Data

↓

Template Updates

↓

Component Removed

↓

ngOnDestroy()

↓

Cleanup

25. Summary

ngOnChanges()

→ Input changed

ngOnInit()

→ Initialize / load data

ngAfterViewInit()

→ View ready

ngOnDestroy()

→ Cleanup

Then:

HttpClient

→ HTTP communication

provideHttpClient()

→ Configure HTTP

get()

→ Retrieve data

Observable

→ Asynchronous result

subscribe()

→ Receive result

5.7 Template-Driven Forms

1. Connection with 5.6

In 5.6, Angular received data from servers.

"How does Angular collect and validate data entered by the user?"

User

↓

Form

↓

Angular

↓

Validation

↓

Component Logic

Angular provides two approaches:

Angular Forms

↓

↓ ↓

Template-Driven Forms | Reactive Forms

2. Definition: A template-driven form is an Angular form approach in which most form configuration is defined in the HTML template using Angular directives. Angular automatically creates and manages the corresponding form-control objects.

Template-Driven Form

→ Form mainly defined in HTML

It is suitable for relatively simple forms such as:

Login

Registration

Contact

Feedback

3. Need for Template-Driven Forms

A registration form may need:

Student Name

Email

Department

Year

Terms Agreement

Submit Button

Angular must handle:

User Input

↓

Data Binding

↓

Validation

↓

Form State

↓

Submission

4. Definition: FormsModule provides Angular directives and functionality required for template-driven forms, including ngModel and NgForm.

Import:

```
import {
  FormsModule
} from '@angular/forms';
```

Standalone component:

```
@Component({
  imports: [
    FormsModule
  ]
})
```

FormsModule

→ Enables Template-Driven Forms

5. Definition: ngModel is an Angular forms directive used to create/manage a form control and can synchronize its value with a component property.

Common syntax:

```
<input
  [(ngModel)]="student.name">
```

Working:

Component Property

↕

[(ngModel)]

↕

Input Control

6. Why name is Important

Inside a template-driven <form>, controls using ngModel normally require a name.

Example:

```
<input
  type="text"
  name="studentName"
  [(ngModel)]="student.name">
```

Memory:

name

→ Registers / identifies
control inside form

7. Binding Multiple Form Values

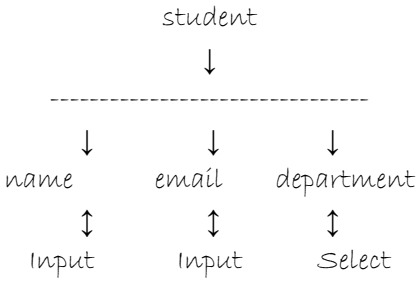
Component:

```
student = {
  name: "",
  email: "",
  department: ""
};
```

Template:

```
<input
  name="studentName"
  [(ngModel)]="student.name">

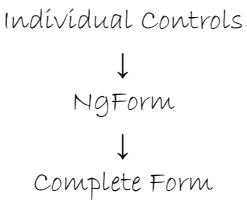
<input
  name="email"
  [(ngModel)]="student.email">
```



8. Definition: NgForm is an Angular directive that represents and manages the complete template-driven form, including its controls, values, validity, and state.

Syntax:

```
<form
  #studentForm="ngForm">
</form>
```



9. Definition: A template reference variable is a name declared using # that gives access to an element, directive, component, or form object inside the template.

Form example:

```
<form
  #studentForm="ngForm">
```

Control example:

```
<input
  name="studentName"
  [(ngModel)]="student.name"
  #nameControl="ngModel">
```

Now:

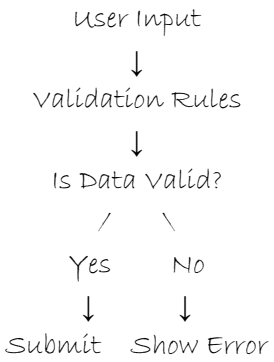
studentForm

→ Complete form state

nameControl

→ Individual control state

10. Definition: Form validation checks whether user-entered values satisfy specified rules before the application accepts or processes them.



11. Common Validation Rules

required

→ Value must be entered

email

→ Must satisfy email format

minlength

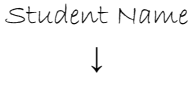
→ Minimum characters required

maxlength

→ Maximum characters allowed

Example:

```
<input
  type="text"
  name="studentName"
  [(ngModel)]="student.name"
  required
  minlength="3">
```



Required

+

Minimum 3 Characters

12. Email Validation

```
<input
  type="email"
  name="email"
  [(ngModel)]="student.email"
  required
  email>
```

Requirements:

Value Present

+

Valid Email

13. Form-Control States

Angular automatically tracks important states:

State	Meaning
valid	Validation rules pass
invalid	One or more rules fail
touched	User focused and left field
untouched	Field not yet left after focus
dirty	User changed value
pristine	User has not changed value

valid / invalid

→ Is value correct?

touched / untouched

→ Has user visited field?

dirty / pristine

→ Has user changed value?

14. Displaying Validation Message

```
<input
  name="studentName"
  [(ngModel)]="student.name"
  required
  minlength="3"
  #nameControl="ngModel">

<p
  *ngIf="
    nameControl.invalid &&
    nameControl.touched ">
```

Please enter a valid name.
</p>

flow:

User Visits Field
↓
Leaves Invalid Value
↓
touched = true
invalid = true
↓
Show Error Message

15. Definition: ngSubmit is an Angular forms event used to handle form submission and call a component method.

Syntax:

```
<form
  (ngSubmit)="onSubmit()">
</form>
Component:
onSubmit(): void {

  console.log(
    "Form Submitted"
  );
}
```

16. Passing NgForm to Component

Template:

```
<form
  #studentForm="ngForm"
  (ngSubmit)="onSubmit(studentForm)">
Component:
onSubmit(
  form: NgForm
): void {

  console.log(
    form.value
  );
}
```

Working:

Submit Button
↓
ngSubmit

↓

onSubmit(studentForm)

↓

Component Receives

Complete Form

17. Checking Form Validity

```
<button
  type="submit"
  [disabled]="studentForm.invalid">

  Register

</button>
```

Flow:

All Controls

↓

Validation

↓

NgForm

↓

Form Valid?

/ \

Yes No

↓ ↓

Enable Disable

Submit Submit

18. Radio Button Example

```
<label>
  <input
    type="radio"
    name="year"
    value="First Year"
    [(ngModel)]="student.year">

  First Year
</label>

<label>
  <input
    type="radio"
    name="year"
    value="Second Year"
    [(ngModel)]="student.year">
```

```
  Second Year
</label>
Same:
name
+
model property
allows one value to be selected.
```

19. Checkbox Example

```
acceptedTerms: boolean =
  false;
<input
  type="checkbox"
  name="acceptedTerms"
  [(ngModel)]="acceptedTerms"
  required>
```

Use:

```
Terms Accepted?
      ↓
true / false
```

20. Resetting the Form

NgForm provides:

```
form.resetForm();
```

Example:

```
onSubmit(
  form: NgForm
): void {

  console.log(
    form.value
  );

  form.resetForm();
}
```

Definition: **resetForm() resets registered controls and the form state.**

21. Diagram

```
FormsModule
      ↓
HTML Template
      ↓
   <form>
      ↓
ngModel Controls
      ↓
```

name Registers Controls

↓

Validation Rules

↓

NgForm

↓

Form Valid?

↓

ngSubmit

↓

Component Method

22. Scenario — Student Registration

Requirements:

Student Name

→ Required + Minimum 3 Characters

Email

→ Required + Valid Email

Department

→ Required

Year

→ Select One

Terms

→ Must Accept

Register Button

→ Enabled only if form is valid

Flow:

Student Enters Data

↓

[(ngModel)]

↓

Validation

↓

NgForm

↓

Valid Form

↓

ngSubmit

↓

Register Student

23. Example

```
import {
  Component
} from '@angular/core';

import {
  CommonModule
} from '@angular/common';

import {
  FormsModule,
  NgForm
} from '@angular/forms';

@Component({
  selector:
    'app-registration',

  standalone: true,

  imports: [
    CommonModule,
    FormsModule
  ],

  template: `
    <h2>
      Student Registration
    </h2>

    <form
      #registrationForm="ngForm"
      (ngSubmit)="
        onSubmit(registrationForm)
      ">

      <input
        type="text"
        name="studentName"
        [(ngModel)]="student.name"
        required
        minlength="3"
        #nameControl="ngModel">

      <p
        *ngIf="
          nameControl.invalid &&
```

```
            nameControl.touched
          ">

          Enter a valid name.

        </p>

        <br>

        <input
          type="email"
          name="email"
          [(ngModel)]="student.email"
          required
          email>

        <br><br>

        <button
          type="submit"
          [disabled]="
            registrationForm.invalid
          ">

          Register

        </button>
      </form>
  `
})
export class RegistrationComponent {

  student = {
    name: "",
    email: ""
  };

  onSubmit(
    form: NgForm
  ): void {

    if (form.valid) {

      console.log(
        form.value
      );
    }
```

```
  }
}
```

24. Template-Driven vs Ordinary HTML Form

Ordinary HTML Form	Angular Template-Driven Form
Collects browser values	Integrates values with Angular
Basic HTML validation	Angular tracks validity
No ngModel	Uses ngModel
No Angular NgForm	Uses NgForm
Manual app handling	Connected to component logic

25. Template-Driven vs Reactive — Preview

Template-Driven	Reactive
Mainly configured in HTML	Mainly configured in TypeScript
Uses FormsModule	Uses reactive-form APIs
Commonly uses ngModel	Uses explicit control objects
Good for simpler forms	Better for more programmatic control

26. Summary

Template-Driven Form

→ Mainly defined in HTML

FormsModule

→ Enables form directives

[(ngModel)]

→ Two-way form binding

name

→ Registers control

NgForm

→ Represents complete form

#variable

→ Access form/control state

ngSubmit

→ Handle submission

required

→ Mandatory value

resetForm()

→ Reset form

Validation states:

valid / invalid

→ validation result

touched / untouched

→ user visited field or not

dirty / pristine

→ user changed value or not

5.8 Reactive Forms, Validation and Routing

1. Connection with 5.7

In 5.7, form configuration was mainly written in HTML.

"What if we need stronger control over the form from TypeScript?"

Template-Driven Form

↓

Mostly HTML

↓

Need More Programmatic Control

↓

Reactive Form

↓

Form Model in TypeScript

Angular routing is also introduced here to move between different application views.

2. Definition: A reactive form is an Angular form approach in which the form model, controls, values, and validation rules are explicitly created and managed in the TypeScript component.

Reactive Form

→ Form Model in TypeScript

TypeScript

↓

FormGroup

↓

FormControl

↓

Validators

↓

HTML Template

3. Template-Driven vs Reactive Form

Template-Driven	Reactive
Mainly HTML	Mainly TypeScript
FormsModule	ReactiveFormsModule
ngModel	FormControl
NgForm	FormGroup
Validation in template	Validators in TypeScript
Good for simpler forms	Greater programmatic control

Template-Driven

→ HTML controls form

Reactive

→ TypeScript controls form

4. Definition: ReactiveFormsModule provides the Angular functionality and directives required to build reactive forms.

Import:

```
import {
  ReactiveFormsModule
} from '@angular/forms';
```

Standalone component:

```
@Component({
  imports: [
    ReactiveFormsModule
  ]
})
```

Memory:

ReactiveFormsModule

→ Enables Reactive Forms

5. Definition: A FormControl represents one individual form field and tracks its value, validation status, and interaction state.

Example:

```
studentName =
  new FormControl('');
```

Connection to input:

```
<input
  type="text"
  [formControl]="studentName">
```

HTML Input

↕

FormControl

↕

Value + Validation + State

6. Definition: A FormGroup combines several related FormControl objects and represents them as one complete form.

Example:

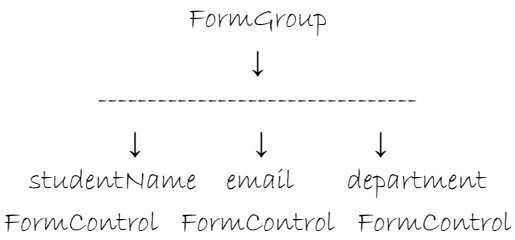
```
registrationForm =
  new FormGroup({

    studentName:
      new FormControl(''),

    email:
      new FormControl(''),
```

```
  department:
    new FormControl('')
});
```

diagram:



F. Connecting FormGroup to HTML

Form:

```
<form
  [formGroup]="registrationForm">
```

Individual controls:

```
<input
  formControlName="studentName">

<input
  formControlName="email">
```

Memory:

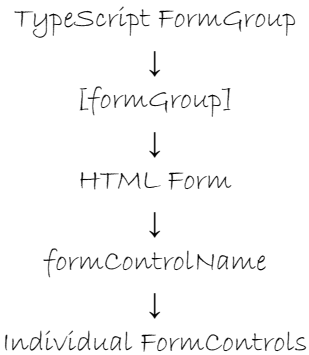
[formGroup]

→ Connect complete form

formControlName

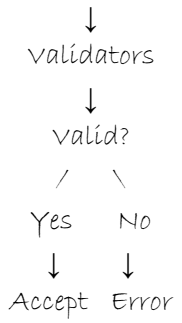
→ Connect individual control

diagram:



8. Definition: Validation is the process of checking whether user-entered data satisfies predefined requirements before it is accepted or processed.

User Input



9. Definition: Validators is an Angular class that provides built-in validation functions for reactive forms.

Important validators:

Validators.required

→ Mandatory value

Validators.email

→ Valid email

Validators.minLength(3)

→ Minimum 3 characters

Validators.maxLength(...)

→ Maximum length

Example:

```
studentName:
  new FormControl(
    '',
    [
      Validators.required,
      Validators.minLength(3)
    ]
  )
```

10. Form-Control States

valid

→ All validations pass

invalid

→ Validation fails

touched

→ User visited and left field

untouched

→ Not yet left

dirty
→ User changed value

pristine
→ Value not changed

Validation message example:

```
registrationForm
  .controls
  .studentName
  .hasError('required')
```

and:

```
invalid + touched
      ↓
Show Error Message
```

11. Form Submission

Template:

```
<form
  [formGroup]="registrationForm"
  (ngSubmit)="onSubmit()">
```

Component:

```
onSubmit(): void {

  if (
    this.registrationForm.valid
  ) {
    console.log(
      this.registrationForm.value
    );
  }
}
```

Disable button when invalid:

```
<button
  type="submit"
  [disabled]="registrationForm.invalid">

  Register

</button>
```

12. Reactive Form Working Principle

```
ReactiveFormsModule
        ↓
TypeScript Component
        ↓
```

FormGroup

↓

FormControls

↓

Validators

↓

[formGroup]

+ formControlName

↓

User Input

↓

Validation

↓

ngSubmit

13. Definition: Angular routing is the mechanism that associates browser URL paths with Angular components and displays the appropriate component when navigation occurs.

Example:

/

→ HomeComponent

/register

→ RegistrationComponent

/about

→ AboutComponent

14. Definition: A route is a configuration that maps a URL path to a component or routing behaviour.

Example:

```
export const routes: Routes = [

  {
    path: '',
    component: HomeComponent
  },

  {
    path: 'register',
    component:
      RegistrationComponent
  }
];
```

Path
→ Component

15. Definition: provideRouter() configures Angular Router services using the application's route definitions in a modern standalone application.

Example:
```
providers: [
  provideRouter(routes)
]
```
Working:

routes
↓
provideRouter()
↓
Angular Router

16. Definition: routerLink is an Angular directive used to navigate to configured routes from a template.

Example:
```
<a routerLink="/">
  Home
</a>

<a routerLink="/register">
  Registration
</a>
```
Flow:

User Clicks Link
↓
routerLink
↓
Angular Router
↓
Matching Route

17. Definition: RouterOutlet is the placeholder where Angular displays the component activated by the current route.

Syntax:
```
<router-outlet>
</router-outlet>
```

Example:

URL = /register
↓

RegistrationComponent

↓

router-outlet

↓

Registration View Displayed

18. Routing Diagram

User Selects Link

↓

routerLink

↓

Angular Router

↓

Match URL Path

↓

Activate Component

↓

router-outlet

↓

Display View

19. Default and Wildcard Routes

Default route:

```
{
  path: '',
  component: HomeComponent
}
```

Wildcard route:

```
{
  path: '**',
  component:
    NotFoundComponent
}
```

Memory:

''

→ Default Route

**

→ Unknown / Unmatched Route

Wildcard is normally placed last.

20. Website Scenario — College Portal

Home

↓

User Selects Registration

↓

routerLink="/register"

↓

Angular Router

↓

RegistrationComponent

↓

Reactive Form

↓

FormGroup

↓

FormControls

↓

Validators

↓

Valid Form

↓

Submit

This combines **Reactive Forms + Validation + Routing**.

21. Combined Reactive Form Example

```
registrationForm =
  new FormGroup({

    studentName:
      new FormControl(
        '',
        [
          Validators.required,
          Validators.minLength(3)
        ]
      ),

    email:
      new FormControl(
        '',
        [
          Validators.required,
          Validators.email
        ]
      ),

    department:
      new FormControl(
        '',
        Validators.required
      )
  });
```

Template:

```
<form
  [formGroup]="registrationForm"
  (ngSubmit)="onSubmit()">

  <input
    formControlName="studentName">

  <input
    type="email"
    formControlName="email">

  <select
    formControlName="department">

    <option value="">
      Select Department
    </option>

    <option value="CSE">
      CSE
    </option>

  </select>

  <button
    type="submit"
    [disabled]="registrationForm.invalid">

    Register

  </button>

</form>
```

22. Summary

Reactive Form

→ Form model in TypeScript

ReactiveFormsModule

→ Enables reactive forms

FormControl

→ One field

FormGroup

→ Group / complete form

Validators

→ Validation rules

[formGroup]

→ Connect form

formControlName

→ Connect individual control

Routing:

Routing

→ Navigation between views

Routes

→ Define URL paths

provideRouter()

→ Configure Router

routerLink

→ Navigate

router-outlet

→ Display active component